

Course Management Project — Deep Dive Report

This document is for **your eyes only** to prepare you for your project discussion. It explains exactly how everything in your full-stack application works, from the backend database to the frontend UI, so you can confidently answer any questions the Doctor or TA throws at you.

1. High-Level Architecture

Your project uses a standard modern web architecture:

- **Backend:** ASP.NET Core Web API using C#. It acts as the "brain" and data manager.
- **Database:** SQLite, managed through Entity Framework Core (an ORM - Object Relational Mapper).
- **Frontend:** React (Single Page Application). It runs entirely in the user's browser, fetching and displaying data from the backend via HTTP requests (using Axios).

2. The Backend (ASP.NET Core API)

Important Files & Folders

- **Program.cs:** The entry point of the app. This is where services are registered (Dependency Injection), the database is connected, CORS is configured (allowing React on port 3000 to talk to the API on port 5238), and JWT authentication is initialized.
- **Models/:** Contains the C# classes that represent your database tables (Course, Student, Instructor, Enrollment, User). Entity Framework uses these to create the database schema.
- **Data/AppDbContext.cs:** The Entity Framework class that bridges your C# models to the actual SQLite database. It contains DbSet<T> properties for each model.
- **DTOs / (Data Transfer Objects):** These are "flat" classes used to send data between the frontend and backend. We use DTOs instead of raw Models to prevent "over-posting" (security) and avoid infinite loops when models reference each other (e.g., a Course has Students, a Student has Courses).
- **Services/:** This is where the *business logic* lives. Instead of the Controller talking directly to the database, the Controller calls a Service, and the Service talks to the database. This makes the code cleaner and easier to maintain.
- **Controllers/:** These are the API endpoints. They handle incoming HTTP requests (GET, POST, PUT, DELETE), check if the user is authorized using [Authorize], call the appropriate Service, and return JSON responses.

How Authentication Works (Backend)

1. When a user logs in via `AuthController.Login`, the `AuthService` verifies their password.
2. If correct, it generates a **JWT (JSON Web Token)**. This token contains a "claim" with the user's Role (Admin, Instructor, or Student).
3. The backend returns this token (along with a Refresh Token) to the frontend.
4. For any protected endpoint (e.g., `[Authorize(Roles = "Admin")]`), the backend intercepts the request, reads the JWT from the Authorization header, verifies its digital signature, and checks if the role matches.

3. The Frontend (React)

Important Files & Folders

- **src/index.js & App.js:** The entry point. `App.js` uses `react-router-dom` to define all the routes (URLs) in the application. It also holds the global user state.
- **src/services/api.js:** A centralized Axios instance pointing to `http://localhost:5238/api`.
 - **Crucial Detail:** It has a "Request Interceptor". Before any Axios request is sent, this interceptor reads the token from `document.cookie` and attaches it as `Authorization: Bearer <token>`. This is how the backend knows who is making the request.
- **src/components/ProtectedRoute.jsx:** A wrapper component. If a user tries to access `/courses` but the user state is null, it forces a `<Navigate to="/login" />`.
- **src/pages/:** Contains the React components for each screen.
 - **List Pages (e.g., `CoursesListPage.jsx`):** Uses the `useEffect` hook to trigger an API call to fetch all data the moment the component mounts. The data is stored in a `useState` array and mapped into an HTML table.
 - **Form Pages (e.g., `CourseFormPage.jsx`):** Handles both Creation and Editing. It checks if there's an `:id` in the URL. If yes, it fetches the existing data and pre-fills the form. When submitted, it either calls `.create()` or `.update()`.

How Authentication Works (Frontend)

1. The user types their credentials in `LoginPage.jsx`.
2. Axios sends a POST request to `/api/Auth/login`.
3. The backend responds with `{ token, refreshToken, user }`.
4. The frontend saves these three items into **Cookies** (`document.cookie`). We use cookies because they persist even if the user refreshes the page or closes the tab.
5. `App.js` updates its user state variable, which instantly unlocks the `ProtectedRoute` components, allowing the user to see the Dashboard.

4. Deep Dive: The Data Flow of a Single Action

What exactly happens when an Admin clicks "Delete" on a Course?

1. **Frontend Event:** The user clicks the Delete button in `CoursesListPage.jsx`.
2. **Confirm:** A `window.confirm` popup asks "Are you sure?".
3. **API Call:** If they click OK, the `handleDelete(id)` function is fired. It calls `courseService.delete(id)`.
4. **Axios Interception:** The `api.js` interceptor pauses the request, grabs the JWT from the cookie, adds it to the headers, and sends a DELETE `/api/Courses/{id}` HTTP request.
5. **Backend Routing:** ASP.NET receives the request and routes it to `CoursesController.Delete(int id)`.
6. **Backend Auth:** The `[Authorize(Roles = "Admin")]` attribute kicks in. It checks the JWT. Since the token says "Role: Admin", the request is allowed.
7. **Service Layer:** The controller calls `_courseService.DeleteAsync(id)`.
8. **Database Layer:** The service finds the course in the `AppDbContext`, removes it, and calls `SaveChangesAsync()`. This executes a SQL DELETE command in SQLite.
9. **Response:** The backend returns an HTTP 204 No Content status (meaning success, nothing to return).

10. **Frontend Update:** Back in React, the `.then()` block executes. It uses `setCourses(prev => prev.filter(c => c.id !== id))` to instantly remove the deleted course from the screen without refreshing the page.
-

5. Potential Discussion Questions & Answers

Q: Why didn't you use Redux or Context API for state management?

A: "Because the application's state requirements are relatively simple. The only global state is the authenticated user object. Prop drilling from `App.js` was sufficient and keeps the architecture lightweight and easy to debug without the overhead of Redux."

Q: How are you handling security and preventing normal students from deleting courses?

A: "Security is handled in two layers. First, the UI hides the delete button if `user.role !== 'Admin'`. But UI hiding isn't real security. The real security is in the backend `CoursesController`, where the `[Authorize(Roles = "Admin")]` attribute guarantees that even if a student uses Postman or edits the HTML to trigger a delete request, the backend will reject it with a 403 Forbidden error."

Q: What are DTOs and why did you use them?

A: "Data Transfer Objects. They prevent over-posting vulnerabilities (where a user might try to update a field they shouldn't, like an ID). They also solve circular reference issues in JSON serialization when Entity Framework models have relationships (like a Course having Students and Students having Courses)."

Q: Explain how the React Router works in your app.

A: "We use `BrowserRouter` in `App.js`. We define `Routes` that map URL paths to specific React components. For dynamic pages like editing, we use URL parameters like `/courses/:id`. The `CourseFormPage` uses the `useParams` hook to read that ID and fetch the correct data."

Q: Why did you use `useEffect`?

A: "`useEffect` is used to perform side effects in functional components. In my list pages, I use it with an empty dependency array `[]` so that it fires exactly once when the component first mounts, triggering the Axios call to fetch data from the API to populate the tables."